Escola Politècnica Superior

Master in Intelligent Field Robotic Systems (IFRoS) /
Master in Intelligent Robotic Systems (MIRS)

Lab 1: Generation and Uses of ArUco Markers with OpenCV

Students:
Huu Truong Giang Nguyen *(u6110634@udg.edu)*
Duc Phu Nguyen *(u6111090@udg.edu)*

March 9, 2026

Contents

1 Introduction

2 Part 2: Generation of ArUco Markers

2.1 Background & Objectives

ArUco markers are square graphical representation composed of a binary matrix, each of which cell is either black or white, that encodes a unique identifier. The patterns of the matrix are engineered to be mutually distinguishable even under partial occlusion or image noise, and are organized into predefined dictionaries. ArUco markers are widely used in computer vision applications such as camera calibration, augmented reality, and robot navigation, due to their robustness and ease of detection under varying lighting and perspective conditions.
The objective of this part is to generate valid ArUco markers and multi-marker boards using OpenCV's ArUco module. Two command-line programs were developed for this purpose: **generate_marker** and **generate_board**.

2.2 Usage Guide

2.2.1 generate_marker

This program generates a single ArUco marker and saves it as a PNG image. It is executed from the `/build` directory as follows:

```
./generate_marker <dictionary> <id> <size> <output_file>
```

Parameter	Type	Description
dictionary	string	ArUco dictionary name (e.g. DICT_ARUCO_ORIGINAL)
id	integer	Marker ID within the dictionary
size	integer	Marker size in pixels
output_file	string	Name of the output PNG file

Table 1: Parameters for generate_marker

Example command line: `./generate_marker DICT_ARUCO_ORIGINAL 25 200 marker.png`

2.2.2 generate_board

This program generates a grid board of ArUco markers and saves it as a PNG image. It is executed as follows:

```
./generate_board <rows> <cols> <dictionary> <size> <separation> <output_file>
```

Parameter	Type	Description
rows	integer	Number of marker rows in the grid
cols	integer	Number of marker columns in the grid
dictionary	string	ArUco dictionary name
size	integer	Individual marker size in pixels
separation	integer	Gap between adjacent markers in pixels
output_file	string	Name of the output PNG file

Table 2: Parameters for generate_board

Example: `./generate_board 2 4 DICT_ARUCO_ORIGINAL 200 80 board.png`

2.3 Implementation Approach

2.3.1 Marker Generation

The synthesis of ArUco markers must strictly adhere to the predefined dictionaries provided by the OpenCV library to ensure compatibility with the detection pipeline. The dictionary is selected when the user executes the program. Once the dictionary is loaded, the marker is rendered into a grayscale image matrix at the requested pixel size. A white border is then added around the marker. The purpose of which is allowing the ArUco detectors to locate markers by searching for dark square contours against a lighter surrounding region, so the border ensures sufficient contrast at the marker edges.

2.3.2 Board Generation

Generating a board requires laying out multiple markers consistently in a grid. The total image size should be computed from the number of markers, their individual size, and the desired separation to appropriately facilitate the required dimension of the ArUco board. The image dimensions are calculated as follows:

$$W = N_{cols} \times S_{marker} + (N_{cols} - 1) \times S_{sep} + S_{offset} \tag{1}$$

$$H = N_{rows} \times S_{marker} + (N_{rows} - 1) \times S_{sep} + S_{offset} \tag{2}$$

where $S_{offset} = 50$ pixels is a fixed margin added to all sides. This margin serves the same purpose as the individual marker border which allows the edge markers of the board detectable. The ArUco board is constructed using OpenCV's `GridBoard` object, which internally assigns

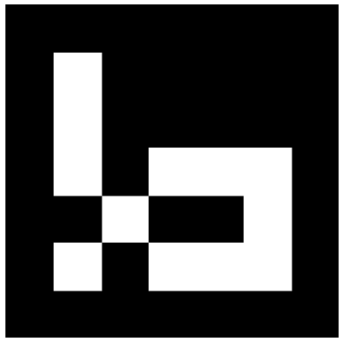

Figure 1: Caption

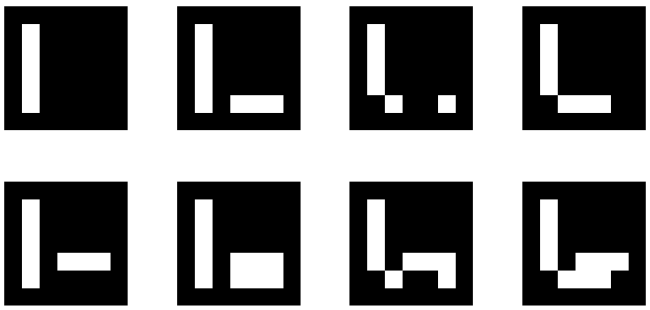

Figure 2: Caption

sequential IDs to each marker position and handles the spatial layout. By default, the first $N_{cols} * N_{rows}$ IDs from the selected dictionary are assigned to the grid positions. Moreover, rendering the board to an image is then handled by the built-in method **generateImage** of the GridBoard object, which places each marker at its computed position with the specified separation.

2.4 Results and Discussion

Figure 1 shows the generated ArUco marker and Figure 2 shows the resulting board. Both outputs were verified to be detectable by OpenCV's ArUco detector. The white borders are clearly visible in both cases, confirming that the padding was applied correctly. The board layout matches the specified grid dimensions, with uniform separation between markers.

3 Part 3: Dectection of ArUco Markers

3.1 Background & Objectives

The foundational step for applications such as Augmented Reality (AR) or camera calibration is the detection and spatial identification of ArUco markers within a scene. OpenCV provides a dedicated module that facilitates the localization, identification, and visualization of these markers. Using predefined dictionaries of binary patterns, the system can distinguish between various markers even under challenging lighting or perspective conditions.

The objective of this section is to implement a C++ utility, `detecting_aruco.cpp`, designed to process a live video stream from a webcam. The program identifies any markers present in the frame and provides a real-time visual overlay indicating their respective IDs and corner orientations.

3.2 Usage Guide

This program performs real-time detection of ArUco markers and renders their identification data onto the video stream. Note that when detecting multiple markers simultaneously, all markers must belong to the same ArUco dictionary type. The program is executed from the `/build` directory using the following syntax:

```
./detecting_aruco <dictionary>
```

Parameter	Type	Description
`dictionary`	string	ArUco dictionary name (e.g., `DICT_6X6_250`)

Table 3: Parameters for `detecting_aruco`

Example command line:
```
./detecting_aruco DICT_6X6_250
```

3.3 Implementation Approach

The implementation begins by initializing the ArUco detector using a specific dictionary selected by the user. This dictionary serves as a reference for the detection algorithm to decode the binary patterns found within the video frames. During the execution of the main processing loop, the algorithm performs two primary tasks: marker detection and visualization.

The detection pipeline yields a collection of 2D image coordinates representing the four corners of each detected marker, along with their unique identifiers (IDs). Using this data, the program utilizes OpenCV's drawing functions to generate a visual feedback overlay. This identification includes:

- A **green contour** tracing the perimeter of each marker.
- A **red dot** positioned at the top-left corner (index 0) to indicate the marker's orientation.
- A **numerical label** displaying the decoded ID of the marker.

This visualization serves as a verification step to ensure that the markers are being correctly tracked before proceeding to pose estimation or calibration.

3.4 Results and Discussion

We conducted an experiment using the DICT_4X4_1000 dictionary with ID 1. As seen in Fig. 3, the ArUco marker was successfully detected; the visualization includes a red dot at the top corner, a green contour tracing the perimeter, and the marker ID which is 1 here.

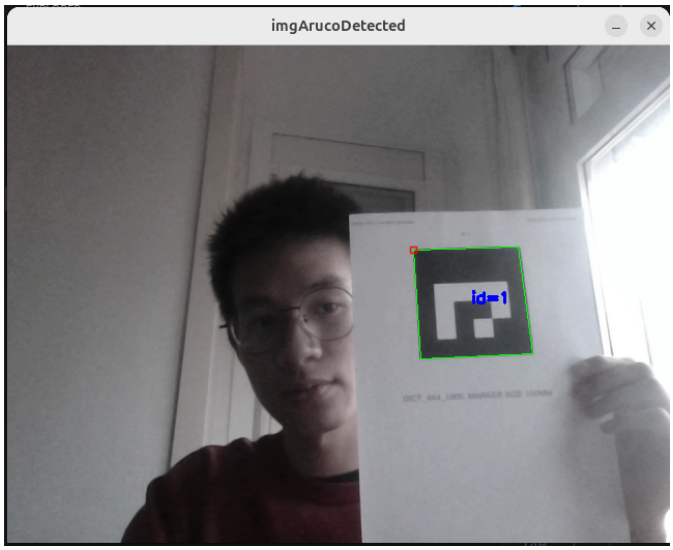

Figure 3: Image capture for detecting ArUco marker

4 Part 4: Camera calibration using an ArUco board

4.1 Backgrounds and Objectives

Camera calibration is an estimation of variables that hold a mathematical relationship between the 3D points and their projected points onto the 2D image plane. The relationship is described by this projective transformation of the 3D points to a 2D image plane following a pinhole camera

model:

$$s\begin{bmatrix}u\\v\\1\end{bmatrix}=\begin{bmatrix}f_x & 0 & c_x\\0 & f_y & c_y\\0 & 0 & 1\end{bmatrix}\begin{bmatrix}r_{11} & r_{12} & r_{13} & t_x\\r_{21} & r_{22} & r_{23} & t_y\\r_{31} & r_{32} & r_{33} & t_z\end{bmatrix}\begin{bmatrix}X_w\\Y_w\\Z_w\\1\end{bmatrix}$$

where

1. $\begin{bmatrix}u\\v\\1\end{bmatrix}$ is the projected 2D point on the homogenous coordinate.
2. $\begin{bmatrix}X_w\\Y_w\\Z_w\\1\end{bmatrix}$ is the 3D point on the homogeneous coordinate.
3. $\begin{bmatrix}f_x & 0 & c_x\\0 & f_y & c_y\\0 & 0 & 1\end{bmatrix}$ is the instrinsic matrix mapping 3D points on the camera coordinate onto the 2D image plane. Parameters such as f_x, f_y are the focal lengths, and (c_x, c_y) is the principal point.
4. $\begin{bmatrix}r_{11} & r_{12} & r_{13} & t_x\\r_{21} & r_{22} & r_{23} & t_y\\r_{31} & r_{32} & r_{33} & t_z\end{bmatrix}$ is the joint rotation-translation matrix or the extrinsic matrix mapping the 3D object point in the world coordinate to the camera coordinate.

4.2 Background and Objectives

Since physical camera systems exhibit geometric distortions—primarily radial and tangential—resulting from lens imperfections, the ideal pinhole camera model must be extended to include distortion coefficients. OpenCV utilizes the Brown-Conrady model to correct projected point coordinates. The mathematical model for the corrected projected point coordinates $[u, v]^T$ is defined as follows:

$$\begin{bmatrix}u\\v\end{bmatrix}=\begin{bmatrix}f_x x'' + c_x\\f_y y'' + c_y\end{bmatrix} \tag{3}$$

where the distorted normalized coordinates $[x'', y'']^T$ are:

$$\begin{bmatrix}x''\\y''\end{bmatrix}=\begin{bmatrix}x'\dfrac{1+k_1r^2+k_2r^4+k_3r^6}{1+k_4r^2+k_5r^4+k_6r^6}+2p_1x'y'+p_2(r^2+2x'^2)+s_1r^2+s_2r^4\\y'\dfrac{1+k_1r^2+k_2r^4+k_3r^6}{1+k_4r^2+k_5r^4+k_6r^6}+p_1(r^2+2y'^2)+2p_2x'y'+s_3r^2+s_4r^4\end{bmatrix} \tag{4}$$

with the radial distance $r^2 = x'^2 + y'^2$ and normalized coordinates:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} X_c/Z_c \\ Y_c/Z_c \end{bmatrix}, \quad \text{if } Z_c \neq 0 \tag{5}$$

In this model, k_n represent the radial coefficients, p_n denote tangential distortion, and s_n account for thin-prism distortion.

Moreover, camera calibration is the process of resolving these parameters by establishing correspondences between known 3D scene points and their 2D projections. ArUco markers provide an efficient framework for this task, acting as a geometric reference where the fixed spatial relationship between markers provides the 3D ground truth. The detection algorithm extracts 2D corner coordinates and utilizes unique marker IDs to ensure unambiguous point matching across different frames.

The objective of this part is to implement a calibration pipeline to estimate the intrinsic, extrinsic, and distortion parameters of a camera. The developed program facilitates real-time ArUco detection, image acquisition, and the subsequent optimization of the camera model based on the captured dataset.

4.3 Usage Guide

The calibration program is executed via the command line from the `/build` directory with the following syntax:

```
./camera_calibrate <dictionary> <detector_params> <num_rows> <num_cols> <marker_length> <se
```

Parameter	Type	Description
`dictionary`	string	ArUco dictionary name (e.g., `DICT_6X6_250`).
`detector_params`	string	Path to detector parameter configuration.
`num_rows`	integer	Number of rows of markers on the board.
`num_cols`	integer	Number of columns of markers on the board.
`marker_length`	float	Side length of a single ArUco marker (meters).
`separation`	float	Gap distance between adjacent markers (meters).
`output_file`	string	Filename for the resulting calibration data (YAML format).

Table 4: Parameters for `camera_calibrate`

Example command:
`./camera_calibrate DICT_6X6_1000 params.yml 2 4 0.05 0.02 calibration_stats`

Operational Procedure: Upon execution, a video stream window displays the real-time identification of detected markers. The user can capture calibration frames by pressing the **'c'** key. Each captured frame is stored in a local `/data` directory (which must be initialized prior to execution). Once a sufficient number of diverse views have been acquired, the program computes the calibration parameters and exports the results to the specified YAML file.

4.4 Implementation Approach

The calibration workflow is divided into three distinct phases: data acquisition, correspondence mapping, and parameter estimation. The process begins by initializing camera streams and loading user-defined parameters. An interactive acquisition loop is then executed, incorporating the ArUco detection scheme detailed in Section 3. During this phase, an integrated capture mechanism allows the user to manually trigger image storage via the keyboard. To ensure a statistically significant sample size for the calibration algorithm, a dataset of 25 distinct images was curated, capturing the board from various orientations and distances.

Following data collection, the system iterates through the stored image set to construct the 2D-3D correspondence lists. For each detected marker, 3D coordinates are defined in the object's local Euclidean space, while their 2D counterparts are extracted from the image plane. These pairs serve as the primary input for the calibration algorithm to solve for the camera's intrinsic matrix, extrinsic parameters, and distortion coefficients.

Regarding the distortion model, the OpenCV implementation adopts the standard Brown-Conrady model, focusing on radial (k_1, k_2, k_3) and tangential (p_1, p_2) components. Given that the hardware utilized is a standard laptop webcam, higher-order radial coefficients (k_4, k_5, k_6) were excluded by disabling the `CALIB_RATIONAL_MODEL` flag. This decision was based on the fact that such parameters are primarily intended for wide-angle or fisheye lenses; applying them to a standard narrow-field lens could lead to numerical instability or overfitting. Furthermore, thin-prism parameters (s_1, s_2, s_3, s_4) were omitted (`CALIB_THIN_PRISM_MODEL` disabled), as the mechanical tilt between the lens and the sensor plane in consumer-grade webcams is typically negligible for general-purpose computer vision tasks.

4.5 Results and Discussion

In this experiment, we captured 25 images of the ArUco board using the `DICT_6X6_1000` dictionary for camera calibration. To ensure data quality, we verified that all markers on the board were detected in every frame, as illustrated in Fig. 4. The dataset consists of 25 images captured from various angles and distances (see Fig. 5 for examples). Following the calibration process, we achieved an acceptable mean reprojection error of approximately 0.993 pixels, as shown in Fig. 6.

The resulting camera matrix and distortion coefficients, saved in a YAML file, are presented in Fig. 7.

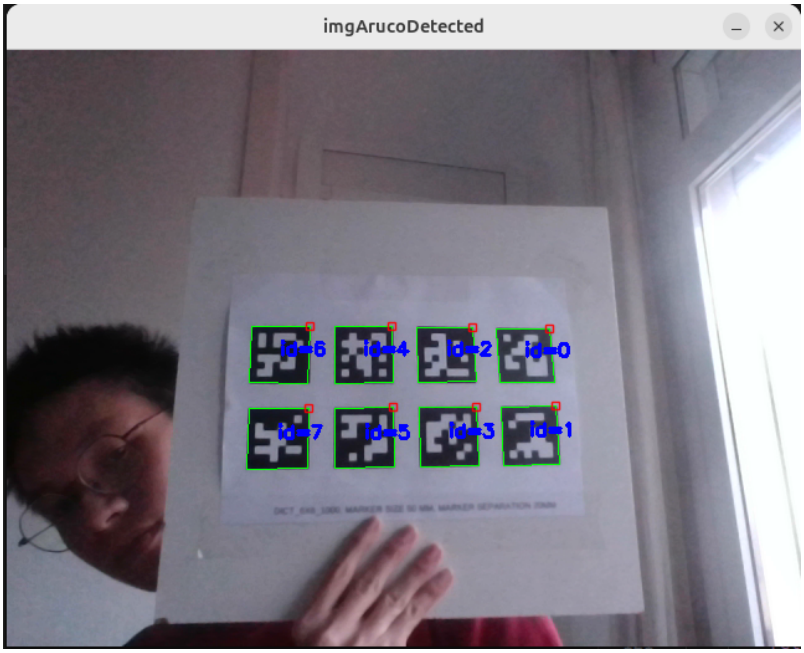

Figure 4: Aruco board Detection

5 Part 5: Using ArUco Markers for Augmented Reality and distance measurements

5.1 Background and Objectives

Utilizing the camera intrinsic matrix and distortion coefficients obtained during the calibration phase, it is possible to estimate the rotation and translation transformations—collectively known as the pose—of a 3D object relative to the camera frame. This estimation requires a set of known 3D object points and their corresponding 2D projections. Such spatial information is foundational for various Augmented Reality (AR) applications, including pose visualization, the rendering of 3D geometries onto physical objects, relative pose estimation and inter-object distance estimation. For instance, virtual 3D points defined in the object's coordinate system can be projected into the 2D image plane using the camera's intrinsic and extrinsic parameters, allowing for seamless visual integration. ArUco markers are particularly well-suited for these tasks, as they provide easily detectable corners with predefined 3D coordinates.

The objective of this section is to implement four distinct programs focusing on different AR applications:

Figure 5: Some example images for calibration

```
giang@giang-Precision-7560:~/IFROS/Hands_on_Perception/lab_1/build$ ./camera_calibrate DICT_6X6_1000 params.yml 2 4 0.05 0.02 calibration_stats
Calibration complete
Reprojection error: 0.993028
Save yaml file to ../config/calibration_stats.yaml
```

Figure 6: Reprojection error

1. Six-degree-of-freedom (6DoF) pose estimation of a single marker.
2. Real-time rendering of a 3D cube onto a detected ArUco marker.
3. Estimate the position of the right ArUco with respect to the left ArUco marker.
4. Calculation of the physical distance between two independent ArUco markers.

5.2 Usage Guide

Four programs all take the same input arguments:

```
./pose_estimation <dictionary> <marker_id> <marker_length>
./draw_cube <dictionary> <marker_id> <marker_length>
./relative_pose_estimation <dictionary> <marker_id> <marker_length>
./distance_estimation <dictionary> <marker_id> <marker_length>
```

```
%YAML:1.0
---
camera_matrix: !!opencv-matrix
   rows: 3
   cols: 3
   dt: d
   data: [ 525.95026247421458, 0., 329.17748883416806, 0.,
       523.69460046001882, 241.01682639368957, 0., 0., 1. ]
distortion_coefficients: !!opencv-matrix
   rows: 1
   cols: 5
   dt: d
   data: [ -0.21264188893467584, 0.99090163941313902,
       0.00085192219156773918, -0.0023838614429069279,
       -1.399515749796217 ]
```

Figure 7: Calibration parameters

Parameter	Type	Description
dictionary	string	ArUco dictionary name (e.g. DICT_6X6_250)
marker_id	numerical	The specific ID of the ArUco marker to be tracked
marker_length	numerical	side length of the ArUco marker in meters

Table 5: Parameters for pose_estimation, draw_cube, relative_pose_estimation, distance_estimation

Example command line: `./pose_estimation DICT_6X6_250 23 0.05`

5.3 Implementation Approach

5.3.1 Pose Estimation (pose_estimation.cpp)

The **pose_estimation** module facilitates the transition from 2D image coordinates to 3D spatial orientation. The process begins by parsing user-defined arguments and loading the camera intrinsic matrix and distortion coefficients. To establish a reference for the pose solver, four virtual 3D points are defined in the marker's local coordinate system. As illustrated in Table 6, the origin $(0, 0, 0)$ is situated at the geometric center of the marker, with all corners residing on the $Z = 0$ plane.

Marker Corner	**Local 3D Coordinate** (X, Y, Z)
Corner 1 (Top-Left)	$(-L/2, L/2, 0)$
Corner 2 (Top-Right)	$(L/2, L/2, 0)$
Corner 3 (Bottom-Left)	$(-L/2, -L/2, 0)$
Corner 4 (Bottom-Right)	$(L/2, -L/2, 0)$

Table 6: Virtual 3D object points for an ArUco marker of side length L.

Following the detection of the marker's 2D corners in the image plane (as detailed in Section 3), the correspondence between these 2D pixels and the 3D virtual points is established. The

camera pose, comprised of a rotation vector (*rvec*) and a translation vector (*tvec*), is subsequently estimated by solving the Perspective-n-Point (PnP) problem, which minimizes the re-projection error between 3D-2D correspondence pairs. Finally, the estimated extrinsic parameters are used to project the X, Y, Z axes onto the image, providing a visual verification of the object's coordinate frame.

5.3.2 3D Geometry Projection (draw_cube.cpp)

The `draw_cube` program extends the pose estimation workflow by projecting a three-dimensional wireframe onto the detected marker. To construct a cube whose base is perfectly aligned with the marker boundaries, eight virtual vertices are defined in the object's local coordinate system. The bottom four vertices coincide with the marker corners ($Z = 0$), while the top four vertices are translated along the positive Z-axis by a distance equal to the marker length L.

These 3D vertices are then transformed into the image plane using the previously estimated *rvec* and *tvec* via the pinhole camera projection model. By drawing lines between the projected 2D correspondences, the system renders a projected virtual cube on to the image. This demonstrates the ability to integrate virtual volumetric objects into a real-world scene.

5.3.3 Relative Pose Estimation of Two ArUco Markers (`relative_pose_estimation.cpp`)

This module computes the relative position of the right ArUco marker with respect to the left marker. By performing pose estimation, we obtain two translation vectors, $\mathbf{t}_1$ and $\mathbf{t}_2$, for the left and right markers, respectively. These vectors represent the displacement from the camera's optical center to each marker's origin in 3D space.

The relative pose $\mathbf{p}$ of the right marker relative to the left marker is calculated by determining the difference between their translation components:

$$p_x = t_{2x} - t_{1x}$$

$$p_y = t_{2y} - t_{1y}$$

$$p_z = t_{2z} - t_{1z}$$

In our experimental setup, the two ArUco markers were placed on the same plane and aligned along the same horizontal axis. This configuration was chosen to simplify the evaluation of the system's accuracy and to allow for straightforward comparison with manual hand measurements.

5.3.4 Distance Estimation of Two ArUco markers (distance_estimation.cpp)

This module computes the Euclidean distance between two independent ArUco markers within the same field of view. By performing pose estimation for both markers, we obtain two translation

vectors, t_1 and t_2, representing the displacement from the camera's optical center to each marker's origin in 3D space.

The Euclidean distance d is calculated based on their translation components:

$$d = \sqrt{(t_{1x} - t_{2x})^2 + (t_{1y} - t_{2y})^2 + (t_{1z} - t_{2z})^2} \tag{6}$$

Nevertheless, in the experimental settings, the students attach two markers on a common planar surface for a better measurement of the distance of two ArUco markers. Therefore, the student assumes the translational terms in z-axis is 0, which results in the distance are computed only by the translational terms in x and y axes.

To visualize the results, the 2D centroid of each marker is calculated by averaging its corner coordinates. A line is then drawn between these centroids, annotated with the calculated physical distance in meters.

5.4 Results and Discussion

5.4.1 Pose estimation

As shown in Fig. 8, we successfully detected the ArUco marker and estimated its pose, visualized by the x, y, and z axes attached to the marker center. The specific pose estimation values are displayed in the top-right corner of the frame.

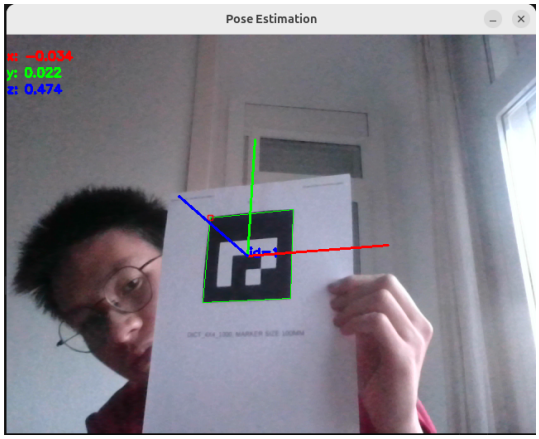

Figure 8: Pose estimation result

5.4.2 3D Geometry Projection

In this experiment, we successfully projected a 3D cube onto the marker, as illustrated in Fig. 9. The results demonstrate that the 3D cube remains correctly projected and aligned even when the marker is moved or rotated.

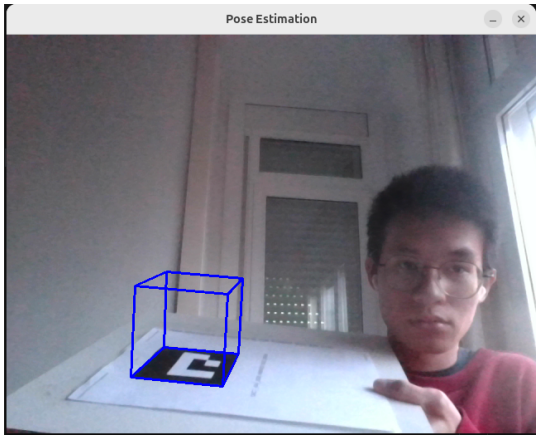

Figure 9: Visulization of project 3D Cube in ArUco marker

5.4.3 Relative Pose Estimation

As seen in Fig. 10, we successfully computed the relative position of the right ArUco marker with respect to the left marker and rendered the coordinate axes for both. The two markers were placed on the same platform, approximately along the same horizontal line. The program yielded accurate results: the y and z displacements are approximately 0, while the x position is 17.6 centimeters. This closely matches our manual measurement of 17.3 centimeters.

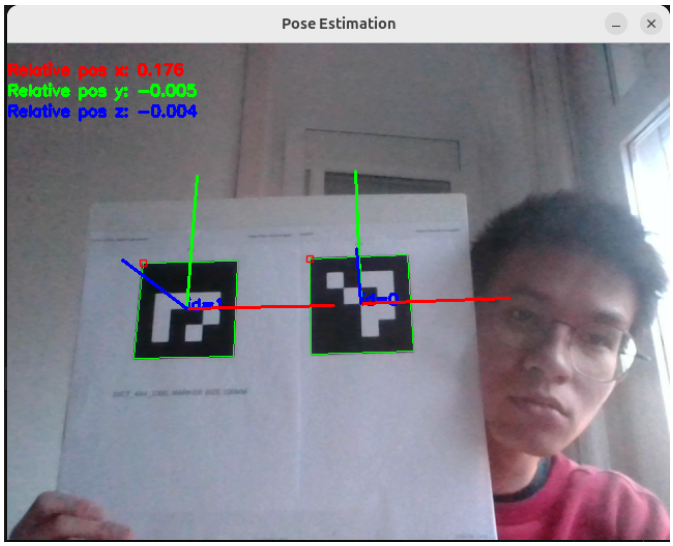

Figure 10: Relative pose estimation result

5.4.4 Distance Estimation between Two ArUco Markers

This experiment involved measuring the Euclidean distance between two ArUco markers. To simplify manual verification, both markers were placed on the same plane at a measured distance of approximately 17.3 centimeters. As shown in Fig. 11, the program draws a straight line between the markers and displays a calculated distance of approximately 17.6 centimeters in the top-right corner. This estimation is considered acceptable, as the discrepancy likely stems from manual measurement inaccuracies and environmental noise.

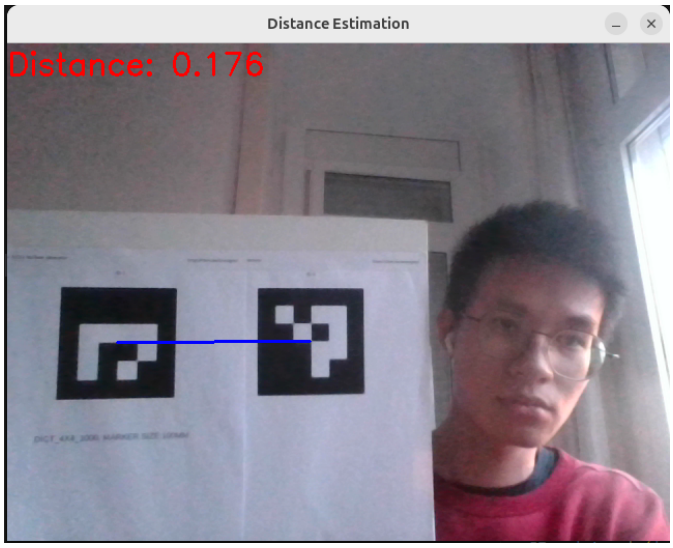

Figure 11: Distance Estimation between two ArUco Markers

6 Conclusion

Throughout the laboratory sessions, we have successfully completed all tasks, ranging from ArUco marker generation to the pose and distance estimation of multiple markers. This lab provided a comprehensive understanding of how to utilize markers not only for camera calibration but also for augmenting 3D space and estimating real-world positions. These capabilities are critical in the field of robotics, where vision-based object pose estimation is a fundamental requirement. By attaching ArUco markers to physical objects, we can efficiently and accurately track their positions and orientations in 3D space, providing a robust solution for robotic perception and manipulation.